Quantstamp

Hemi - YieldVault

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Yield Vault
Timeline	2025-11-17 through 2025-11-20
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Specification	None
Source Code	• vesperfi/yield-vault #c988391
Auditors	• Cameron Biniamow Auditing Engineer • Hytham Farah Auditing Engineer • Andrei Stefan Auditing Engineer

Documentation quality	Low
Test quality	Medium
Total Findings	14 Fixed: 6 Acknowledged: 7 Mitigated: 1
High severity findings	0
Medium severity findings	1 Mitigated: 1
Low severity findings	9 Fixed: 4 Acknowledged: 5
Undetermined severity findings	0
Informational findings	4 Fixed: 2 Acknowledged: 2

Summary of Findings

The Hemi yield vault is an ERC4626 vault that allows users to deposit assets to mint a proportional number of shares. The vault owner, keepers, and maintainers configure the vault with strategies and set desired debt ratios. Vault strategies are expected to frequently rebalance (borrow assets from the vault up to their credit limit or repay outstanding debt) and report profits and losses to the vault. As strategies generate profits, the vault's total assets grow, thereby increasing share prices for holders. Profits generate fees for the reporting strategies, calculated by the duration between rebalances and the profit amount. Alternatively, strategy losses decrease share prices and do not generate any fees. Vault keepers manage the operational state and can pause or shut down the vault, controlling users' ability to mint or redeem shares.

During the review, the audit team uncovered several vulnerabilities in the smart contracts, particularly relating to the operational mechanics of profit and loss reporting, asset management, and governance. One medium-severity finding (HEMI-1) identifies an MEV vulnerability in which an attacker can sandwich the profit reported by a strategy with mint and redeem transactions to capture the increased share price. Nine low-severity issues were identified, most of which have a high impact but low probability of occurring, as the attacks would need to be performed by the strategy owners, who are intended to be the Hemi team. HEMI-2 allows strategies to manipulate fees by strategically reporting profits to dilute users' share price. HEMI-3 reveals an edge case in which simultaneous reporting of profits and losses results in an inflated reduction in the debt ratio for the reporting strategy. HEMI-4 explains the multiple cases where strategies can bypass their debt ratio limits and incur outstanding debt. HEMI-5 describes the lack of enforcement on strategies to repay outstanding debt, allowing strategies to report profits and generate fees while their debt ratio exceeds the limit. The remaining low-severity issues are mainly related to potential configuration errors or inconsistent access controls. The informational issues and auditor suggestions address minor misconfigurations and code improvements to ensure adherence to best practices.

Update: During the fix review, the client fixed, mitigated, or acknowledged all issues listed in the report. Vulnerabilities HEMI-2, HEMI-3, HEMI-5, HEMI-10, HEMI-12, and HEMI-14 have been remediated effectively. HEMI-1 is effectively mitigated, as the client will report earnings at high frequency or use Flashbots to minimize MEV opportunities. However, vulnerabilities HEMI-4, HEMI-6, HEMI-7, HEMI-8, HEMI-9, HEMI-11, and HEMI-13 have been acknowledged, with the client providing rationale for their design choices regarding these items or a low probability of exploitation. Additionally, suggestions S1 through S5 have been implemented and fixed, while S6 remains acknowledged with an explanation on its handling of strategy and withdrawal processes.

ID	DESCRIPTION	SEVERITY	STATUS
HEMI-1	MEV Opportunity on Profit Reporting in `YieldVault`	• Medium	Mitigated

ID	DESCRIPTION	SEVERITY	STATUS
HEMI-2	Fee Manipulation via Timestamp Gaming	Low	Fixed
HEMI-3	Simultaneous Profit and Loss Reporting Edge Case	Low	Fixed
HEMI-4	Bypass of Debt Ratio Limits	Low	Acknowledged
HEMI-5	Bypassable Debt Pay Back	Low	Fixed
HEMI-6	Keeper Can Remove All Other Keepers and Achieve Unilateral Control	Low	Acknowledged
HEMI-7	Owner Can Set Fee to 100% of Profits	Low	Acknowledged
HEMI-8	Strategy Can Report Earnings While Vault Is Shutdown	Low	Acknowledged
HEMI-9	Excess Debt and Credit Limit Calculations Use Incorrect Total Assets Amount when Reporting Profits	Low	Acknowledged
HEMI-10	`updateWithdrawQueue()` Does Not Check for Duplicate Strategies	Low	Fixed
HEMI-11	Decimal Offset Immutability Issue With Upgradeable Asset Tokens	Informational	Acknowledged
HEMI-12	Decimal Offset Calculation Can Underflow for High-Decimal Tokens	Informational	Fixed
HEMI-13	Minimum Deposit Limit Set to 1 Wei	Informational	Acknowledged
HEMI-14	Incorrect Storage Location Hash	Informational	Fixed

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following

1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.

2. Testing and automated analysis that includes the following:
   1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
   2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

Repo: `https://github.com/vesperfi/yield-vault`

Included Paths: `src/`

Operational Considerations

1. **Strategy Debt and Credit Management**: The vault allocates assets to strategies based on configured debt ratios with dynamic credit line calculation. Each strategy has a debt ratio (expressed in basis points out of 10,000) that determines its maximum debt allocation. When strategies report earnings, the vault calculates the available credit line as the minimum of: the strategy's remaining debt allocation, the vault's total remaining debt allocation, and the vault's available asset balance. Strategies receive additional assets if the credit line exceeds profit and payback, or return assets if payback exceeds the credit line. This creates operational dependency on strategies properly implementing withdrawal functionality and maintaining asset balances for reporting.
2. **Universal Fee Time-Based Accrual**: The vault charges universal fees on strategy profits based on time elapsed since last rebalance multiplied by strategy debt. Fee calculation: `(universalFee * (now - lastRebalance) * totalDebt) / (MAX_BPS * ONE_YEAR)`. This creates scenarios where strategies accumulate higher fees the longer they wait between reports, incentivizing infrequent `reportEarning()` calls. Fee is capped at `maxProfitAsFee` percentage of actual profit, which can fully consume small profits. The `lastRebalance` timestamp is updated on every `reportEarning()` call, regardless of profit amount.
3. **Withdrawal Queue and Strategy Dependencies**: User withdrawals exceeding the vault's immediate balance trigger sequential withdrawal from strategies in withdrawal queue order. Each strategy in the queue is called via a try-catch block, silently continuing to the next strategy on failure. Debt accounting is reduced based on actual assets received rather than the requested amount. If all strategies fail or return insufficient assets, withdrawal reverts with the `AssetsCanNotBeWithdrawn` error. This creates an operational requirement for strategies to maintain sufficient liquidity and properly implement the `withdraw()` function, or risk blocking user withdrawals.
4. **Vault Upgradability and Owner Preservation**: The vault uses the UUPS proxy pattern with owner-authorized upgrades. The `upgradeToAndCall()` function includes explicit owner validation before and after upgrade, reverting if the owner address changes. This prevents malicious upgrade implementations from hijacking vault ownership. However, the validation only checks the owner address, not the full vault state, including keeper/maintainer lists, strategy configurations, or asset balances.
5. **Keeper and Maintainer Self-Management**: Keepers can add and remove other keepers, including themselves, from the keeper list. If a keeper removes all other keepers and then removes themselves, only the owner can perform keeper functions. The same applies to the maintainer list management. The owner is not automatically in the keeper/maintainer lists after initialization (they are added explicitly), but can always perform these functions due to modifier checks: `msg.sender != owner() && !isKeeper(msg.sender)`.
6. **ERC4626 Compliance and Share Calculation**: The vault implements the ERC4626 standard with decimal offset calculation for tokens with decimals other than 18. Offset is calculated once during initialization as `18 - asset.decimals()`. All share calculations use this offset for proper decimal handling. The `totalAssets()` calculation includes both the vault's direct balance and all strategy debt: `totalDebt + vaultBalance`. This means the share price fluctuates based on strategy performance even without explicit profit reporting.
7. **Strategy Balance Check Timing**: The `reportEarning()` function validates the strategy has a sufficient balance for reported profit and payback: `balanceOf(strategy) < (profit + payback)`. However, this check occurs before the actual asset transfers. If the strategy's balance decreases between the check and the transfer (e.g., due to reentrancy or external calls), `safeTransferFrom()` may fail. The function uses OpenZeppelin's SafeERC20, which will revert on transfer failure, but the upfront check provides early validation.
8. **Factory Vault Creation Permissionlessness**: The `YieldVaultFactory` allows anyone to call `createVault()` with arbitrary parameters, including name, symbol, asset, and owner. While this enables permissionless vault creation, it also allows creation of vaults with malicious assets, misleading names, or attacker-controlled owners. The factory maintains no registry or validation of created vaults. Users must independently verify vault legitimacy before depositing.
9. **Pause vs Shutdown State Interactions**: The vault implements both `PausableUpgradeable` (pause) and `ShutdownableUpgradeable` (shutdown) states. Paused state blocks deposits and mints via the `whenNotPaused` modifier, but allows withdrawals and redeems. Shutdown state blocks withdrawals and redeems via `whenNotShutdown` modifier, but allows strategy operations. The `paused()` function returns true if either paused **or** shutdown is active. Cannot unpause while shutdown is active. This creates four operational states: normal, paused, shutdown, and paused+shutdown, requiring careful state management. Users have no way to exit their positions until a keeper calls `restart()`. If keeper keys are lost or keepers become uncooperative, funds are permanently locked.
10. **Strategy Withdrawal Debt Accounting**: When withdrawing from strategies, debt is reduced by a minimum of the strategy's current debt and actual assets withdrawn. This handles cases where the strategy returns less than requested or has recorded debt but insufficient assets. However, if strategy returns **more** than its recorded debt (e.g., through external profits), the excess is added to the vault balance, but debt is

only reduced by the recorded debt amount, potentially creating accounting discrepancies where `totalAssets()` includes unreported strategy profits.

11. **Strategy Asset Management**: Strategies are completely trusted with vault assets and must honestly report profits and losses through `reportEarning()` - the vault cannot force-withdraw from strategies, and malicious strategies can brick withdrawals by reverting.
12. **Vault Creation Upgradability**: The factory owner can update the vault implementation at any time via `updateImplementation()`. When users call `createVault()`, they may believe they're deploying a trusted vault implementation, but if the owner frontruns the call or has already set a malicious implementation, the newly created vault will use that malicious code.
13. **Out of Scope Contracts**: The strategy contracts are out of scope and unknown to the audit team. Vulnerabilities in the strategy contracts could result in the misuse of profit/loss reporting and potentially lead to the loss of user funds.

Key Actors And Their Capabilities

1. **Users**: Can deposit assets into the vault to receive shares according to the ERC4626 standard, subject to minimum deposit limit validation. Users can mint shares by specifying a desired share amount. Users can withdraw assets or redeem shares at any time when the vault is not shut down, with withdrawals potentially pulling funds from strategies in the withdrawal queue order. Users can transfer their vault shares to other addresses. User deposit and withdrawal operations are blocked when the vault is paused, but they can still transfer shares.
2. **Owner**: The vault owner has ultimate authority over vault configuration and strategy management. The owner can add new strategies with specified debt ratios (the percentage of vault assets allocated to each strategy). The owner can remove strategies only when they have zero debt. The owner can configure the universal fee percentage (default 2%), the maximum profit as a fee cap (default 50%), and the minimum deposit limit. The owner can set the vault rewards contract address for external reward distribution. The owner can sweep accidentally sent ERC20 tokens (except the vault's asset token). The owner authorizes all vault upgrades through the UUPS proxy pattern. The owner serves as the default keeper and maintainer upon initialization.
3. **Keepers**: Manage vault operational state and access control with significant emergency powers. Can pause and unpause vault to block deposits and mints during emergencies. Can shut down the vault, which blocks all withdrawals and redeems, forces strategies to return all debt, and can only be reversed by a keeper restart. Can add and remove other keepers and maintainers from their respective lists. The owner is automatically added as a keeper during initialization and can always perform keeper actions regardless of their membership in the keeper list.
4. **Maintainers**: Handle ongoing strategy parameter adjustments and fund allocation optimization. Can update debt ratio for active strategies to rebalance fund allocation, with total debt ratio across all strategies not exceeding 100%. Can reorder the withdrawal queue to optimize withdrawal efficiency and strategy priority. Cannot add or remove strategies, only adjust parameters of existing active strategies. Owner can always perform maintainer actions regardless of maintainer list membership.
5. **Strategies**: External contracts that receive vault assets to generate yield through various DeFi protocols. Must implement `IStrategy` interface, including `withdraw()`, `feeCollector()`, and vault functions. Call `reportEarning()` to return profits, report losses, and pay back excess debt beyond their debt ratio allocation. Strategies receive assets from the vault when reporting earnings if they have an available credit line based on the debt ratio. Report standalone losses via the `reportLoss()` function for immediate loss events like deposit fees. Strategies must maintain sufficient asset balance to cover reported profits and paybacks or transaction reverts. Receive assets automatically during `reportEarning()` when the credit line exceeds profit plus payback amounts.
6. **Fee Collectors**: Addresses specified by each strategy's `feeCollector()` function that receive universal fee shares. Universal fees are calculated based on the strategy's total debt, time since last rebalance, and the vault's universal fee rate. The fee is capped at the maximum profit as a fee percentage (default 50% of the reported profit). Fee shares are minted directly to the fee collector during profit reporting without asset transfer. No validation that the fee collector is a non-zero address, potentially burning fees if the strategy returns `address(0)`.
7. **Vault Rewards**: Optional external contract that receives notifications on every share mint, burn, and transfer via updateReward calls. Used for external reward distribution mechanisms like staking rewards or loyalty programs. No validation of vault rewards contract implementation, trusts external call will not revert or consume excessive gas.

Findings

HEMI-1 MEV Opportunity on Profit Reporting in `YieldVault`

Medium | Mitigated

Update

The client mitigated the issue and provided the following explanation:

```
Rebalance or reportEarning() is called by keeper . To avoid MEV on Ethereum mainnet, flashbot can be
used or frequency of rebalance can be increased.
```

File(s) affected: `src/YieldVault.sol`

Description: The `YieldVault.reportEarning()` flow allows MEV strategies to capture strategy profits by sandwiching profit reports with deposits and withdrawals. Share price changes are applied immediately within the same block, with no deposit delay, withdrawal lock, or slippage check.

Exploit Scenario:

1. Attacker calls `deposit()` with large amount → mints shares at current price
2. Strategy calls `reportEarning(profit, 0, 0)` → increases `totalAssets()` → share price increases
3. Attacker calls `redeem()` in same block → burns shares at new higher price → captures profit

4. Net result: Attacker extracts value from existing LPs who earned the profit

Recommendation: Consider vesting profits frequently and in small amounts to prevent MEV attacks.

HEMI-2 Fee Manipulation via Timestamp Gaming

Low Fixed

Update
Fixed by the client in commit `2b7c03ab9c1a81652460a91dfcb099ccbda53848`.

File(s) affected: `src/YieldVault.sol`

Description: The fee calculation uses `block.timestamp - _config.lastRebalance` and updates `lastRebalance` **after** calculating fees. This enables two opposing attacks:

1. *Fee Evasion* - Strategy reports 1 wei profit (resets timestamp), then immediately reports real profit with ~0 time elapsed → pays almost no fees.
2. *Fee Maximization* - Strategy delays reporting to maximize `(timeElapsed × totalDebt)` product → LPs pay excessive fees (up to 50% of profits) and LPs get unfairly diluted.

Recommendation: Consider making fees depend solely on profits and not on time elapsed.

HEMI-3 Simultaneous Profit and Loss Reporting Edge Case

Low Fixed

Update
Fixed by the client in commit `3bc333f07f5fedd3e2d6aa04f852278139aeda50`.

File(s) affected: `src/YieldVault.sol`

Description: The `reportEarning()` function allows strategies to report both `profit > 0` and `loss > 0` in the same transaction, which is conceptually unusual as a strategy should not simultaneously gain and lose value. More importantly, this creates a mathematical inconsistency in debt ratio calculations because the loss is processed before profit assets are transferred to the vault.

When `_reportLoss()` is called at `YieldVault#L316`, it calculates the debt ratio reduction using the current `totalAssets()`:

```
uint256 _deltaDebtRatio = Math.min((loss_ * MAX_BPS) / totalAssets(), _config.debtRatio);
```

However, at this execution point:

- Profit has been recorded and fee shares minted (lines 306-310)
- Loss calculation uses `totalAssets = totalDebt + vaultBalance`
- Profit assets haven't been transferred yet (happens at line 340)
- The `totalAssets` value is artificially low because it includes the debt reduction from the loss but excludes the incoming profit

This causes the debt ratio reduction to be calculated against a lower `totalAssets` base, resulting in a disproportionately large penalty to the strategy's future allocation capacity. The strategy debt ratio decreases more than it should, given the actual economic outcome. The strategy hurts itself by reporting both profit and loss simultaneously. It receives more debt ratio reduction than it should, permanently reducing its future allocation capacity. This reduced debt ratio is stored and used for all future credit line calculations, meaning the strategy will receive fewer assets in subsequent rebalancing operations. Only the maintainer can restore the debt ratio via `updateDebtRatio()`, which the strategy has no control over. Vault depositors are not affected as they receive the net economic outcome (profit minus loss) regardless of the calculation ordering.

Recommendation: Prevent simultaneous profit and loss reporting by adding a validation check:

```
if (profit_ > 0 && loss_ > 0) revert CannotReportProfitAndLossSimultaneously();
```

HEMI-4 Bypass of Debt Ratio Limits

Low Acknowledged

Update
The client acknowledged the issue and provided the following explanation:

```
This is not an issue as strategies are managed by vault team only. Strategies may have higher fund than
debt ratio based on user's deposit and withdraw activities. Strategy can not use this fund unless a
rebalance is called . During rebalance method call it will automatically return fund.
```

File(s) affected: `src/YieldVault.sol`

Description: A strategy (or colluding address) can momentarily inflate `totalAssets()` with a flash-loan deposit, trigger `_availableCreditLimit()` inside `reportEarning()`, borrow extra funds, then immediately withdraw the deposit.

Similarly, an honest user could redeem a large number of shares, which would withdraw all available assets from the vault and immediately result in all strategies incurring outstanding debt.

Exploit Scenario:
Flashloan

Concrete example (20% debt ratio, 100k TVL, 20k already lent):

1. Attacker deposits 50k via flash loan → `totalAssets` becomes 150k.
2. `reportEarning()` sees a new debt ceiling of 30k and lends the strategy an extra 10k.
3. Attacker redeems the 50k deposit; idle assets cover the withdrawal, so strategy debt stays at 30k.
4. After withdrawal, `totalAssets` returns to 100k, but the strategy remains 10k over its limit (30% vs. the intended 20%).

Large Withdraw

- Vault has 100k TVL
- Strategy 1: 15% debt ratio, 15k already lent
- Strategy 2: 10% debt ratio, 10k already lent
- User: Holds shares worth 20k

1. User redeems all shares, receives 20k of assets from the vault.
2. The `totalAssets()` of the vault is 80k.
3. Strategy 1 debt ratio is 15/80k → 18.75%; Strategy 2 debt ratio is 10/80k → 12.5%.
4. Both strategies have outstanding debt.

Recommendation: Consider the following recommendations:
1. Depending on the chain and gas costs, consider implementing functionality that rebalances all strategies when shares are redeemed.
2. Add a publicly exposed function that allows admins/users to rebalance a strategy.

HEMI-5 Bypassable Debt Pay Back

Low Fixed

> **Update**
> Fixed by the client in commit `56d754eed853d6f4c4cb6e9e704ba3ba64f98266`.

File(s) affected: `src/YieldVault.sol`

Description: Strategies can incur outstanding debt after share redemption or if their debt ratio is lowered due to reported losses or by protocol admins after the strategy has already borrowed assets from the vault. Strategies are expected to repay their outstanding debt; however, there is no mechanism to enforce repayment. Strategies can repeatedly report profits and accrue fees while outstanding debt remains unpaid.

Recommendation: When strategies report profits, but have outstanding debt, use the profits to pay back the debt first. Alternatively, implement logic similar to the `_withdrawFromStrategy()` function that forcibly withdraws assets from the strategy to reduce its outstanding debt.

HEMI-6
Keeper Can Remove All Other Keepers and Achieve Unilateral Control

Low Acknowledged

> **Update**
> The client acknowledged the issue and provided the following explanation:
>
> ```
> Owner is by-default keeper and maintainer . Even if all keepers are removed , owner can add keeper. We do not want add burden on owner for keeper management.
> ```

Description: Any keeper can add or remove other keepers (including themselves) through `addKeeper()` and `removeKeeper()` functions. A compromised or malicious keeper can remove all other keepers, add themselves as the only keeper, and gain unilateral control over critical vault operations, including pause, shutdown, and keeper management.

Recommendation: Consider some of the following:
1. Require multi-sig approval for keeper removal
2. Restrict keeper management to the owner only
3. Prevent removal of the last keeper

HEMI-7 Owner Can Set Fee to 100% of Profits

Low | Acknowledged

Update

The client acknowledged the issue and provided the following explanation:

> `This is owner only functions. Owner is trusted and multisig. Owner wont set fee 100%`

File(s) affected: `src/YieldVault.sol`

Description: The `updateMaximumProfitAsFee()` function allows the owner to set `_maxProfitAsFee` up to 10,000 (100%), enabling complete extraction of all strategy profits as fees. This leaves LPs with zero returns despite the strategy's successful performance.

Recommendation: Add a reasonable upper bound to prevent excessive fees. Standard DeFi performance fees range from 10% to 20%. Even the current default of 50% is unusually high.

HEMI-8 Strategy Can Report Earnings While Vault Is Shutdown

Low | Acknowledged

Update

The client acknowledged the issue and provided the following explanation:

> `We may want strategy to be wound down when pool shutdown. This process is for getting fund back in pool. If we stop strategy to rebalance when pool shutdown, then admin can remove all keepers of some specific strategies to stop rebalances`

File(s) affected: `src/YieldVault.sol`

Description: The `reportEarning()` function does not check if the vault is in shutdown state before processing earnings. While credit line calculation returns 0 during shutdown (preventing new debt), strategies can still report profits and losses, mint fee shares, and update debt accounting. This creates an inconsistent state where the vault is "shutdown" but strategy operations continue.

Recommendation: Add a shutdown check at the beginning of `reportEarning()` or clearly document that strategies should use `reportLoss()` only during shutdown.

HEMI-9 Excess Debt and Credit Limit Calculations Use Incorrect Total Assets Amount when Reporting Profits

Low | Acknowledged

Update

The client acknowledged the issue and provided the following explanation:

> `This is as designed. Strategy get debt ratio from profit in next rebalance.`

File(s) affected: `src/YieldVault.sol`

Description: When a strategy reports a profit, the calculations for excess debt and available credit limit refer to `totalAssets()`, which does not account for the profit. This is because `totalAssets()` calculates the asset amount as the sum of the vault contract's balance and the vault's total debt. Any profits are not treated as additional debt and have not yet been transferred back into the vault. Therefore, the excess debt calculation will report an inflated value, and the available credit limit will report a deflated value. In both cases, the vault will reclaim more assets from the strategy than are needed, leaving the strategy with total debt below its debt limit.

Recommendation: Consider accounting for profits when calculating `totalAssets()` to ensure the strategy reports the correct total debt after calling `reportEarning()`.

HEMI-10 `updateWithdrawQueue()` Does Not Check for Duplicate Strategies

Low | Fixed

Update

Fixed by the client in commit `d02e1e0159568178d09d464fcdcea7baea88bff6`.

File(s) affected: `src/YieldVault.sol`

Description: The `updateWithdrawQueue()` function validates that all strategies in the new queue are active, but does not check for duplicate strategy addresses. This allows the same strategy to appear multiple times in the withdrawal queue, leading to it being called multiple times during withdrawals.

Recommendation: Add duplicate detection in `updateWithdrawQueue()`.

HEMI-11
Decimal Offset Immutability Issue With Upgradeable Asset Tokens

Informational | Acknowledged

> **Update**
> The client acknowledged the issue and provided the following explanation:
>
> ```
> This is unlikely event
> ```

File(s) affected: `src/YieldVault.sol`

Description: The decimal offset is calculated once during initialization and stored immutably: If the underlying asset token is itself upgradeable and its `decimals()` function returns a different value after an upgrade, the vault's decimal offset becomes incorrect, breaking all share calculation logic. The severity is informational since this is a highly unlikely event.

Recommendation: Calculate the offset dynamically, calling the `decimals()` function on the asset token each time it's needed.

HEMI-12
Decimal Offset Calculation Can Underflow for High-Decimal Tokens

Informational | Fixed

> **Update**
> The client fixed the issue in commit `f6685d2dc5ff94cef3cba85b789a9f8c31ac7414`.

File(s) affected: `src/YieldVault.sol`

Description: The `initialize()` function calculates the decimal offset as `18 - asset.decimals()` without checking if asset decimals exceed 18. For tokens with more than 18 decimals, this calculation underflows in Solidity 0.8.30, causing initialization to revert with no clear error message.

```
// YieldVault.sol:141
$._offset = 18 - IERC20Metadata(asset_).decimals();
```

While underflow reverts are better than silent failures, the vault documentation and error messages don't clearly indicate this limitation.

Recommendation: Add explicit validation with clear error message.

HEMI-13 Minimum Deposit Limit Set to 1 Wei

Informational | Acknowledged

> **Update**
> The client acknowledged the issue and provided the following explanation:
>
> ```
> Owner can set minimum deposit limit in future . Currently it is greater than 0 ( 1 wei).  We dont want to allow 0 deposit hence 1 wei is default
> ```

File(s) affected: `src/YieldVault.sol`

Description: The vault sets `_minimumDepositLimit = 1`, which is effectively useless for most assets. For USDC (6 decimals), this is 0.000001 USDC. The limit also checks `assets` rather than `shares`, so as share price increases, the same asset amount yields fewer shares, potentially creating dust positions that round to zero on withdrawal.

Recommendation: Either remove the limit entirely, or set a meaningful value like `1e18` (checking shares) or `1e6` (checking assets for USDC).

HEMI-14 Incorrect Storage Location Hash

Informational | Fixed

> **Update**
> Fixed by the client in commit `d02e1e0159568178d09d464fcdcea7baea88bff6`.

File(s) affected: `src/YieldVault.sol`

Description: The `VaultStorageLocation` hash is incorrectly calculated for:

```
keccak256(abi.encode(uint256(keccak256("vault.storage.YieldVault")) - 1)) & ~bytes32(uint256(0xff))
```

In the contract, the hash is set as `0xbaeaf235c9c9807c1f4a2c352810dd4cb4b0d3d0f2cf0b692f9279e99df38e00`. However, the correct hash is `0xea9b954cc57c3e4e2cc61f2841492d7ea35a35ae7a8ac017d9e469b796800500`.

Recommendation: Update `VaultStorageLocation` to use the correct hash.

Auditor Suggestions

S1 Missing Natspec Documentation for Key Functions

Fixed

> **Update**
> Fixed by the client in commit `965a90c2137cd095849f78faa0cf34976f6a069c`.

File(s) affected: `src/YieldVault.sol`

Description: Several functions lack comprehensive NatSpec documentation, including `_reportLoss()`, `_withdrawFromStrategy()`, `_calculateUniversalFee()`, and `_excessDebt()`. This makes it difficult for auditors and developers to understand the intended behavior and assumptions.

Recommendation: Add detailed NatSpec comments for all internal functions explaining parameters, return values, assumptions, and side effects.

S2 Pausable and Shutdownable State Confusion

Fixed

> **Update**
> Fixed by the client in commit `6ad17e6e5199b15af40bf90e65ac0b7e9a7172df`.

File(s) affected: `src/ShutdownableUpgradeable.sol`

Description: The `paused()` function returns `true` if the contract is either paused **or** shutdown, but the `_unpause()` function requires the contract to **not** be shutdown. This creates a confusing state where users checking `paused()` cannot distinguish between a temporary pause and a permanent shutdown.

Recommendation: Make the state distinction clear.

S3 Dead Code: Proxy File with Import Only

Fixed

> **Update**
> Fixed by the client in commit `42bdcb44f708b323437a990ade651865e513687e`.

File(s) affected: `src/Proxy.sol`

Description: The file contains only a pragma and an unused import. No contract, interface, or library is declared. This adds a compilation unit with no functional purpose.

Recommendation: Remove the file if it is not needed, or add a concrete proxy contract if the file is intended as a wrapper.

S4 Missing Events for Critical Parameter Changes

Fixed

Update

Fixed by the client in commit `ceae95002bd83c77500e25553e5e37b31a0b65bd`.

File(s) affected: `src/YieldVault.sol`

Description: Several functions update critical state variables but don't emit events, making it difficult to track changes off-chain:

1. `addStrategy()` updates `_totalDebtRatio` but only emits `StrategyAdded` (line 385)
2. `removeStrategy()` updates `_totalDebtRatio` but only emits `StrategyRemoved` (line 418)
3. `addKeeper()` / `removeKeeper()` don't emit events (lines 505-515)
4. `addMaintainer()` / `removeMaintainer()` don't emit events (lines 521-531)

While some events are emitted, the keeper/maintainer changes have no events at all.

Recommendation: Add events for all state changes.

S5 Remove Unused Import in `YieldVaultFactory`

Fixed

Update

Fixed by the client in commit `b72c9b7766ba53fc81043872fbca562d88c850ca`.

File(s) affected: `src/YieldVaultFactory.sol`

Description: The `YieldVaultFactory` imports `YieldVault` but does not reference it in executable code. `YieldVault` is only mentioned in NatSpec comments. Proxy creation uses `ERC1967Proxy` with an implementation address and does not require the `YieldVault` type.

Recommendation: Remove the unused import to reduce unnecessary dependencies.

S6 Inconsistent Use of `SafeERC20` for Asset Transfers

Acknowledged

Update

The client acknowledged the suggestion and provided the following explanation:

```
Any given strategy from withdrawQueue, is not bound to withdraw requested amount.
  Strategy can return anywhere between 0 to _assetNeeded. and hence the balanceOf() checks before and
after.
  Strategy may revert at withdraw() and in that case we do not want to fail withdraw in vault and hence
the try catch
```

File(s) affected: `src/YieldVault.sol`

Description: The contract uses `SafeERC20` for some asset transfers (`safeTransfer()`, `safeTransferFrom()` in `reportEarning()`) but relies on `balanceOf()` checks and try-catch in `_withdrawFromStrategy()`. While functional, this creates inconsistency in how transfer failures are handled.

Recommendation: Standardize on `SafeERC20` usage throughout, or document the rationale for different approaches in different contexts.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.

- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/vesperfi/yield-vault`

- `07f...8c3 ./src/Proxy.sol`
- `1a9...43c ./src/ShutdownableUpgradeable.sol`
- `6cc...c29 ./src/YieldVault.sol`
- `e27...817 ./src/YieldVaultFactory.sol`
- `2f2...d74 ./src/interfaces/IStrategy.sol`
- `28b...b60 ./src/interfaces/IVaultRewards.sol`
- `1b1...051 ./src/interfaces/IYieldVault.sol`

Test Suite Results

The test suite was run using `forge test`. All 125 test cases ran successfully.

Update: The test suite was updated to account for the code fixes. All tests passed.

```
[⠒] Compiling...
[⠒] Compiling 86 files with Solc 0.8.30
[⠃] Solc 0.8.30 finished in 3.51s
Compiler run successful!
Analysing contracts...
Running tests...

Ran 23 tests for test/YieldVault.keeper.t.sol:YieldVault_Keeper_Test
[PASS] test_addKeeper() (gas: 64683)
[PASS] test_addKeeper_revertWhen_addingSameKeeperAgain() (gas: 65531)
[PASS] test_addKeeper_revertWhen_callerIsNotKeeper() (gas: 18747)
[PASS] test_addMaintainer() (gas: 64637)
[PASS] test_addMaintainer_revertWhen_addingSameMaintainerAgain() (gas: 65419)
[PASS] test_addMaintainer_revertWhen_callerIsNotKeeper() (gas: 18656)
[PASS] test_pause() (gas: 36983)
[PASS] test_pause_revertWhen_callerIsNotKeeper() (gas: 18065)
[PASS] test_removeKeeper() (gas: 48339)
[PASS] test_removeKeeper_revertWhen_callerIsNotKeeper() (gas: 18636)
[PASS] test_removeKeeper_revertWhen_removingNonExistingKeeper() (gas: 18010)
[PASS] test_removeMaintainer() (gas: 48388)
[PASS] test_removeMaintainer_revertWhen_callerIsNotKeeper() (gas: 18722)
[PASS] test_removeMaintainer_revertWhen_removingNonExistingMaintainer() (gas: 18052)
[PASS] test_restart() (gas: 25575)
[PASS] test_restart_revertWhen_callerIsNotKeeper() (gas: 18087)
[PASS] test_restart_revertWhen_vaultIsNotShutdown() (gas: 14456)
[PASS] test_shutdown() (gas: 34312)
[PASS] test_shutdown_revertWhen_callerIsNotKeeper() (gas: 18127)
[PASS] test_unpause() (gas: 28252)
[PASS] test_unpause_revertWhen_callerIsNotKeeper() (gas: 18042)
[PASS] test_unpause_revertWhen_vaultIsNotPaused() (gas: 17294)
[PASS] test_unpause_revertWhen_vaultIsShutdown() (gas: 37236)
Suite result: ok. 23 passed; 0 failed; 0 skipped; finished in 261.93ms (49.55ms CPU time)

Ran 8 tests for test/YieldVault.maintainer.t.sol:YieldVault_Maintainer_Test
```

```
[PASS] test_updateDebtRatio() (gas: 236719)
[PASS] test_updateDebtRatio_revertWhen_callerItNotMaintainer() (gas: 18853)
[PASS] test_updateDebtRatio_revertWhen_debtRatioIsInvalid() (gas: 228032)
[PASS] test_updateDebtRatio_revertWhen_strategyIsNotActive() (gas: 16135)
[PASS] test_updateWithdrawQueue() (gas: 391160)
[PASS] test_updateWithdrawQueue_revertWhen_arrayLengthMismatch() (gas: 381313)
[PASS] test_updateWithdrawQueue_revertWhen_callerItNotMaintainer() (gas: 19928)
[PASS] test_updateWithdrawQueue_revertWhen_strategyIsNotActive() (gas: 384863)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 264.85ms (54.44ms CPU time)

Ran 4 tests for test/YieldVaultFactory.t.sol:YieldVaultFactoryTest
[PASS] test_createVault_upgradable() (gas: 549027)
[PASS] test_createVault_upgradable_differentAssets() (gas: 1892932)
[PASS] test_updateImplementation() (gas: 7023373)
[PASS] test_updateImplementation_revertWhen_callerIsNotOwner() (gas: 7012209)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 270.69ms (61.03ms CPU time)

Ran 3 tests for test/YieldVault.proxy.t.sol:YieldVault_Proxy_Test
[PASS] test_upgradeToAndCall() (gas: 14725541)
[PASS] test_upgradeToAndCall_revertWhen_calledDirectlyOnImplementation() (gas: 7141901)
[PASS] test_upgradeToAndCall_revertWhen_callerIsNotOwner() (gas: 14689662)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 268.20ms (58.23ms CPU time)

Ran 27 tests for test/YieldVault.owner.t.sol:YieldVault_Owner_Test
[PASS] test_addStrategy() (gas: 236198)
[PASS] test_addStrategy_multiStrategy() (gas: 380651)
[PASS] test_addStrategy_revertWhen_callerIsNotOwner() (gas: 21041)
[PASS] test_addStrategy_revertWhen_debtRatioIsInvalid() (gas: 231733)
[PASS] test_addStrategy_revertWhen_strategyIsActive() (gas: 229143)
[PASS] test_addStrategy_revertWhen_strategyIsNull() (gas: 13713)
[PASS] test_removeStrategy() (gas: 188484)
[PASS] test_removeStrategy_maintainWithdrawQueueOrder() (gas: 423661)
[PASS] test_removeStrategy_revertWhen_callerIsNotOwner() (gas: 17069)
[PASS] test_removeStrategy_revertWhen_debtIsNotZero() (gas: 516452)
[PASS] test_removeStrategy_revertWhen_removingInactiveStrategy() (gas: 31308)
[PASS] test_sweep() (gas: 1061119)
[PASS] test_sweep_revertWhen_callerItNotOwner() (gas: 15443)
[PASS] test_sweep_revertWhen_fromTokenIsAsset() (gas: 18278)
[PASS] test_sweep_revertWhen_receiverIsNull() (gas: 11743)
[PASS] test_updateMaximumProfitAsFee() (gas: 17592)
[PASS] test_updateMaximumProfitAsFee_revertWhen_callerItNotOwner() (gas: 14941)
[PASS] test_updateMaximumProfitAsFee_revertWhen_inputIsHigherThanMaxLimit() (gas: 11454)
[PASS] test_updateMinimumDepositLimit() (gas: 17609)
[PASS] test_updateMinimumDepositLimit_revertWhen_callerItNotOwner() (gas: 14965)
[PASS] test_updateMinimumDepositLimit_revertWhen_inputIsZero() (gas: 11428)
[PASS] test_updateUniversalFee() (gas: 17503)
[PASS] test_updateUniversalFee_revertWhen_callerItNotOwner() (gas: 14986)
[PASS] test_updateUniversalFee_revertWhen_inputIsHigherThanMaxLimit() (gas: 11388)
[PASS] test_updateVaultRewards() (gas: 35189)
[PASS] test_updateVaultRewards_revertWhen_VaultRewardsIsNull() (gas: 11364)
[PASS] test_updateVaultRewards_revertWhen_callerItNotOwner() (gas: 14934)
Suite result: ok. 27 passed; 0 failed; 0 skipped; finished in 265.21ms (55.13ms CPU time)

Ran 14 tests for test/YieldVault.strategy.t.sol:YieldVault_Strategy_Test
[PASS] test_reportEarning() (gas: 608697)
[PASS] test_reportEarning_loss() (gas: 654297)
[PASS] test_reportEarning_payback() (gas: 644813)
[PASS] test_reportEarning_profit() (gas: 667659)
[PASS] test_reportEarning_profitAndPayback() (gas: 680592)
[PASS] test_reportEarning_profitWithUniversalFee() (gas: 715752)
[PASS] test_reportEarning_profitWithUniversalFee2() (gas: 692862)
[PASS] test_reportEarning_revertWhen_strategyHasLessFundThanReporting() (gas: 626460)
[PASS] test_reportEarning_revertWhen_strategyIsNotActive() (gas: 231812)
[PASS] test_reportEarning_vaultIsShutdown() (gas: 552613)
[PASS] test_reportLoss() (gas: 641118)
[PASS] test_reportLoss_revertWhen_lossIsHigherThanDebt() (gas: 227356)
[PASS] test_reportLoss_revertWhen_strategyIsNotActive() (gas: 14052)
[PASS] test_reportLoss_zeroAmount() (gas: 235936)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 3.33s (77.39ms CPU time)

Ran 20 tests for test/YieldVault.t.sol:YieldVault_Test
[PASS] test_deposit() (gas: 332646)
```

```
[PASS] test_deposit_revertWhen_assetsAreLessThanMinimumLimit() (gas: 18464)
[PASS] test_deposit_revertWhen_vaultIsPaused() (gas: 43412)
[PASS] test_deposit_updateRewards() (gas: 348429)
[PASS] test_mint() (gas: 345533)
[PASS] test_mint_revertWhen_assetsAreLessThanMinimumLimit() (gas: 35337)
[PASS] test_mint_revertWhen_vaultIsPaused() (gas: 41189)
[PASS] test_redeem_revertWhen_assetsCanNotBeWithdrawn() (gas: 613851)
[PASS] test_redeem_revertWhen_sharesAreZero() (gas: 14236)
[PASS] test_redeem_revertWhen_vaultIsShutdown() (gas: 58109)
[PASS] test_redeem_success() (gas: 369749)
[PASS] test_redeem_withdrawFromMultipleStrategy() (gas: 1381752)
[PASS] test_redeem_withdrawFromSecondStrategy() (gas: 1018584)
[PASS] test_redeem_withdrawFromStrategy() (gas: 901707)
[PASS] test_redeem_withdrawFromStrategy_errorInTryCatch() (gas: 1130711)
[PASS] test_transfer_updateRewards() (gas: 376121)
[PASS] test_withdraw() (gas: 369364)
[PASS] test_withdraw_revertWhen_assetsAreZero() (gas: 16516)
[PASS] test_withdraw_revertWhen_vaultIsShutdown() (gas: 43552)
[PASS] test_withdraw_updateRewards() (gas: 387684)
Suite result: ok. 20 passed; 0 failed; 0 skipped; finished in 3.42s (111.17ms CPU time)

Ran 26 tests for test/ERC4626.t.sol:ERC4626StdTest
[PASS] test_RT_deposit_redeem((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 555956,
~: 555813)
[PASS] test_RT_deposit_withdraw((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 556090,
~: 555864)
[PASS] test_RT_mint_redeem((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 559539, ~:
559250)
[PASS] test_RT_mint_withdraw((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 559579, ~:
559385)
[PASS] test_RT_redeem_deposit((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 555732,
~: 556111)
[PASS] test_RT_redeem_mint((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 559187, ~:
559534)
[PASS] test_RT_withdraw_deposit((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 555760,
~: 556070)
[PASS] test_RT_withdraw_mint((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 559144, ~:
559470)
[PASS] test_asset((address[4],uint256[4],uint256[4],int256)) (runs: 256, μ: 490911, ~: 491120)
[PASS] test_convertToAssets((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 500121, ~:
500559)
[PASS] test_convertToShares((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 500111, ~:
500600)
[PASS] test_deposit((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 256, μ: 548823, ~:
549001)
[PASS] test_maxDeposit((address[4],uint256[4],uint256[4],int256)) (runs: 256, μ: 491107, ~: 491316)
[PASS] test_maxMint((address[4],uint256[4],uint256[4],int256)) (runs: 256, μ: 491105, ~: 491313)
[PASS] test_maxRedeem((address[4],uint256[4],uint256[4],int256)) (runs: 256, μ: 491260, ~: 491468)
[PASS] test_maxWithdraw((address[4],uint256[4],uint256[4],int256)) (runs: 256, μ: 494668, ~: 494744)
[PASS] test_mint((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 256, μ: 552365, ~:
552388)
[PASS] test_previewDeposit((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 541203, ~:
541136)
[PASS] test_previewMint((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 544835, ~:
544744)
[PASS] test_previewRedeem((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 547894, ~:
547994)
[PASS] test_previewWithdraw((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ: 547984, ~:
548201)
[PASS] test_redeem((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 256, μ: 555945, ~:
555906)
[PASS] test_redeem_zero_allowance((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ:
515173, ~: 517018)
[PASS] test_totalAssets((address[4],uint256[4],uint256[4],int256)) (runs: 256, μ: 492300, ~: 492508)
[PASS] test_withdraw((address[4],uint256[4],uint256[4],int256),uint256,uint256) (runs: 256, μ: 555872, ~:
555871)
[PASS] test_withdraw_zero_allowance((address[4],uint256[4],uint256[4],int256),uint256) (runs: 256, μ:
514905, ~: 517044)
Suite result: ok. 26 passed; 0 failed; 0 skipped; finished in 3.65s (22.69s CPU time)

Ran 8 test suites in 3.70s (11.73s CPU time): 125 tests passed, 0 failed, 0 skipped (125 total tests)
```

Code Coverage

Code coverage was generated with `forge coverage`. The in-scope contracts have moderate to high test coverage. The audit team recommends improving the test suite to achieve branch and statement coverage greater than 90% for `YieldVaultFactory` and function coverage greater than 90% for `YieldVault`.

File	% Lines	% Statements	% Branches	% Funcs
src/ShutdownableUpgradeable.sol	100.00% (32/32)	100.00% (27/27)	100.00% (2/2)	100.00% (12/12)
src/YieldVault.sol	94.64% (265/280)	95.57% (302/316)	96.36% (53/55)	88.14% (52/59)
src/YieldVaultFactory.sol	100.00% (15/15)	82.35% (14/17)	0.00% (0/3)	100.00% (3/3)
test/ERC4626.t.sol	100.00% (9/9)	100.00% (9/9)	100.00% (0/0)	100.00% (1/1)
test/YieldVault.proxy.t.sol	100.00% (4/4)	100.00% (2/2)	100.00% (0/0)	100.00% (2/2)
test/YieldVaultTestBase.t.sol	100.00% (10/10)	100.00% (9/9)	100.00% (0/0)	100.00% (3/3)
test/mocks/MockERC20.sol	100.00% (4/4)	100.00% (2/2)	100.00% (0/0)	100.00% (2/2)
Total	95.76% (339/354)	95.55% (365/382)	91.67% (55/60)	91.46% (75/82)

Changelog

- 2025-11-21 - Initial report
- 2025-12-09 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 500 audits and secured over $200 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Polygon, Avalanche, Curve, Solana, Compound, Lido, MakerDAO, Arbitrum, OpenSea and the World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Near, Flow, Avalanche, Solana, Cardano, Binance Smart Chain, Hedera Hashgraph, Tezos
- DeFi: Curve, Compound, Maker, Lido, Polygon, Arbitrum, SushiSwap
- NFT: OpenSea, Parallel, Dapper Labs, Decentraland, Sandbox, Axie Infinity, Illuvium, NBA Top Shot, Zora
- Academic institutions: National University of Singapore, MIT

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

© 2025 – Quantstamp, Inc.